

STATISTISCHES LERNEN 2

Aula 1

General Linear Models

& Basis Functions

Prof. Johannes Schwab, PhD

FH Kufstein Tirol

Bem-vindo à Aula 1 — General Linear Models & Basis Functions

Esta aula é o ponto de partida de todo o curso. Você aprenderá a formalizar o problema de Aprendizado Estatístico, construir modelos que vão além da reta simples usando **Basis Functions**, e resolver tudo com álgebra linear pura. Ao final, você dominará uma linguagem matemática unificada que serve de base para regularização, bias-variance tradeoff, e muito mais.

Roteiro desta aula:

- 1. O Problema do Aprendizado Estatístico — $Y = f(x) + \epsilon$
- 2. Prediction vs. Inference — dois objetivos completamente diferentes
- 3. Regressão Linear — forma matricial e a Design Matrix
- 4. Normal Equations — solução analítica exata
- 5. Basis Functions — o truque não-linear que mantém a álgebra simples
- 6. Polynomial Basis — potências de x , underfitting e overfitting
- 7. Gaussian RBF Basis — sinos locais com center e width
- 8. Fourier Basis — para dados periódicos
- 9. B-Splines — Compact Support, Partition of Unity, Cox-de Boor

1. O Problema do Aprendizado Estatístico

Imagine que você quer prever o preço de uma casa a partir de sua área. Você tem dados — pares (área, preço) — mas o mundo real é bagunçado: duas casas com a mesma área podem ter preços diferentes por fatores que você não mediu. O **Statistical Learning Problem** formaliza isso de forma elegante: existe uma relação **sistemática** entre entrada e saída, mais um **ruído irreduzível**.

A Equação Fundamental:

$$Y = f(\mathbf{x}) + \varepsilon$$

Y: variável alvo | x: vetor de features | f: função desconhecida | ε: ruído com média zero

O símbolo **f** representa toda a *informação sistemática* que **x** carrega sobre **Y**. Nosso trabalho é **estimar f** a partir de dados observados. O ruído **epsilon** é irreduzível — não importa quão bom seja o modelo, ele sempre estará presente.

Símbolo	Nome	Papel no modelo
Y	Variável alvo (target)	O que queremos prever ou entender
x = (x1, ..., xp)	Vetor de features	Informação disponível (entrada)
f	Função sistemática	Relação real entre x e Y (desconhecida)
epsilon	Ruído irreduzível	Variação que não depende de x

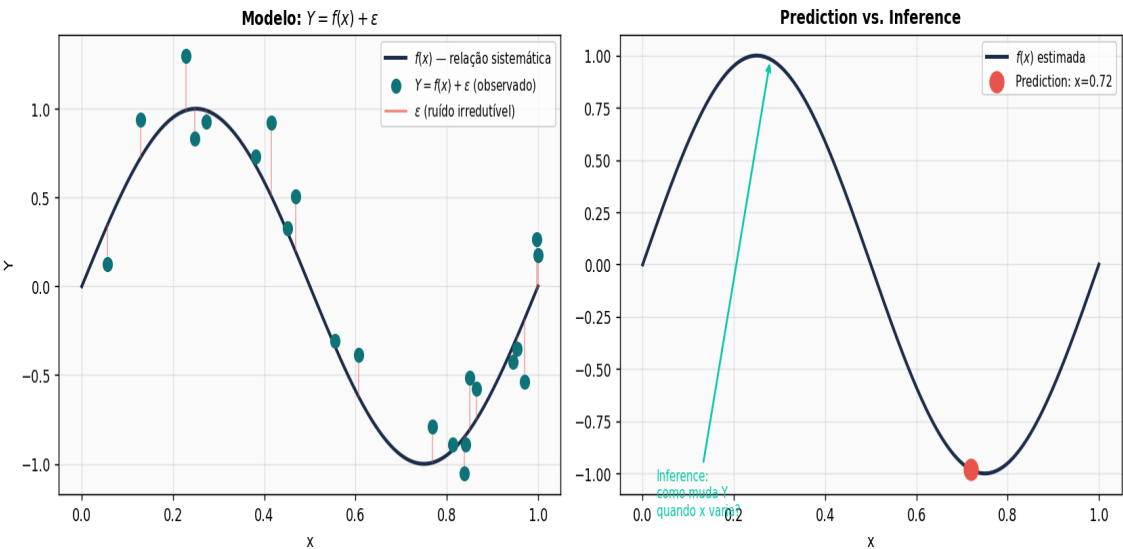


Figura 1 — Esquerda: componentes do modelo $Y = f(x) + \epsilon$. Direita: distinção entre Prediction (ponto vermelho) e Inference (seta verde).

Por que isso é importante para Data Science?

Todo problema de Machine Learning começa aqui. Entender que os dados têm estrutura (**f**) + aleatoriedade (**epsilon**) é o que separa uma análise ingênua de uma abordagem científica. Sem essa formalização, você não consegue quantificar incerteza, comparar modelos, ou entender o que é possível prever.

Aplicação Prática em Machine Learning

Na prática, essa distinção guia a escolha do modelo: para Prediction, modelos complexos (Random Forests, Neural Networks) são aceitáveis mesmo sem interpretabilidade. Para Inference, modelos lineares simples ou GAMs (Generalized Additive Models) são preferidos porque permitem ler os coeficientes como efeitos causais.

2. Prediction vs. Inference

Há dois motivos completamente diferentes para estimar f — e confundi-los leva a escolhas de modelo erradas. **Prediction** quer apenas que $f(x) \approx Y$; a estrutura interna não importa. **Inference** quer entender *como* Y muda quando x aumenta em uma unidade — e para isso o modelo precisa ser interpretável.

Objetivo	Pergunta central	Exemplo real	Modelo típico
Prediction	Dado x novo, qual será Y ?	Prever se um e-mail é spam	Random Forest, Neural Network
Inference	Como Y muda quando x aumenta?	Efeito de 1 ano de educação no salário	Regressão Linear, GAM

Insight crítico: para Prediction, a caixa-preta pode ser opaca. Para Inference, precisamos saber exatamente o que cada coeficiente significa. General Linear Models são ótimos para Inference porque os coeficientes têm interpretação direta.

Por que isso é importante para Data Science?

Essa distinção é fundamental no dia-a-dia de um Data Scientist. Quando um médico quer saber se um medicamento causa melhora (Inference), ele precisa de um modelo interpretável — não apenas de uma previsão acurada. Quando uma Netflix quer recomendar um filme (Prediction), interpretabilidade é secundária; acurácia é o que importa.

Aplicação Prática em Machine Learning

Antes de escolher um algoritmo, sempre se pergunte: o cliente quer uma *previsão* ou quer *entender* o fenômeno? Isso determina se você usa sklearn's RandomForestRegressor (Prediction) ou statsmodels.OLS com coeficientes e p-values (Inference).

3. Regressão Linear — Forma Matricial

A **Regressão Linear** assume que f é linear nos parâmetros: a saída y é uma combinação ponderada das features. Pense como uma receita — o preço de uma casa é R\$ X por m^2 mais R\$ Y por quarto, mais um valor base.

Forma escalar (para uma observação j):

$$y_j = a_0 + a_1x_j + a_2x_j^2 + \dots + a_nx_j^n + \varepsilon_j$$

Cada observação j tem n features; a_0 é o intercepto (bias)

A Design Matrix — forma compacta:

Adicionamos uma coluna artificial de 1s para absorver o intercepto a_0 . Com K observações e $n+1$ coeficientes, empilhamos tudo em uma única equação matricial:

$$\mathbf{y} = \mathbf{X}\mathbf{a} + \varepsilon$$

$\mathbf{y} \in \mathbb{R}^K$ (targets) | $\mathbf{X} \in \mathbb{R}^{(K \times n+1)}$ (Design Matrix) | $\mathbf{a} \in \mathbb{R}^{(n+1)}$ (coeficientes)

A Design Matrix $\mathbf{X}$ tem uma **linha por observação** e uma **coluna por coeficiente**. A primeira coluna é sempre 1 (para o intercepto):

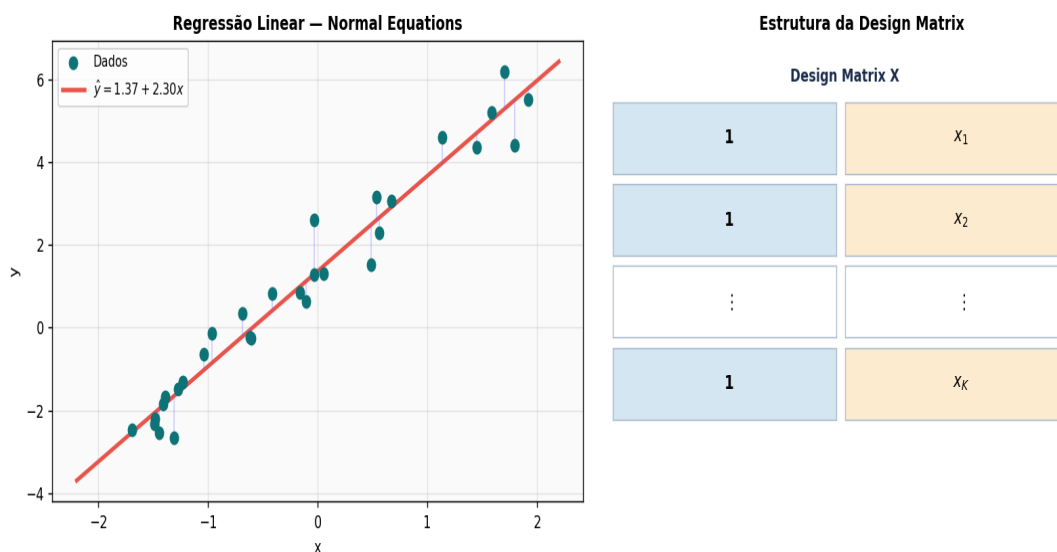


Figura 2 — Esquerda: ajuste linear com Normal Equations (resíduos em roxo). Direita: estrutura da Design Matrix (coluna de 1s + coluna de features).

Por que isso é importante para Data Science?

A forma matricial generaliza para qualquer número de features sem mudar a equação. Com 1 feature ou com 1000 features, o modelo ainda é $y = X \cdot a + \text{epsilon}$. Isso torna o código extremamente genérico — você escreve uma vez e funciona para qualquer dimensão.

Aplicação Prática em Machine Learning

Em Python, você construiu a Design Matrix com `np.column_stack([np.ones(n), x])`. Em sklearn, o `PolynomialFeatures` faz isso automaticamente para você. Entender a estrutura interna permite depurar bugs, implementar variantes customizadas, e entender o que o modelo realmente está otimizando.

4. Normal Equations — Solução Analítica Exata

Queremos encontrar os coeficientes $\hat{a}$ que minimizem a **soma dos erros ao quadrado** (Least Squares): $\|y - X \cdot a\|^2$. A derivação é surpreendentemente elegante: derivamos em relação a a , igualamos a zero, e obtemos a solução analítica exata.

Derivação das Normal Equations:

- Minimizar: $L(a) = \|y - Xa\|^2 = (y - Xa)^T (y - Xa)$
- Derivar: $dL/da = -2X^T(y - Xa) = 0$
- Rearranjar: $X^T X a = X^T y \rightarrow$ (sistema linear em a)
- Solução: $\hat{a} = (X^T X)^{-1} X^T y$

$$\hat{a} = (X^T X)^{-1} X^T y$$

Esta é a solução analítica exata — não uma aproximação iterativa!

np.linalg.solve vs. np.linalg.inv:

Abordagem	Código Python	Estabilidade
Inversão explícita (ruim)	<code>a = np.linalg.inv(X.T@X) @ X.T @ y</code>	Baixa — amplifica erros numéricos
<code>solve</code> (recomendado)	<code>a = np.linalg.solve(X.T@X, X.T@y)</code>	Alta — usa fatoração LU/Cholesky
<code>lstsq</code> (mais robusto)	<code>a = np.linalg.lstsq(X, y)[0]</code>	Máxima — usa SVD, lida com rank-deficient

Por que isso é importante para Data Science?

A solução analítica exata é uma vantagem enorme: não há hiperparâmetros de otimização, não há convergência, não há learning rate. Para datasets de tamanho moderado, é sempre preferível ao Gradient Descent porque garante o mínimo global em uma única operação.

Aplicação Prática em Machine Learning

Sempre use `np.linalg.solve(X.T@X, X.T@y)` em vez de `np.linalg.inv(X.T@X) @ X.T@y`. A inversão explícita acumula erros de ponto flutuante que podem tornar os coeficientes instáveis quando $X^T X$ está próxima de ser singular (o que acontece com grau polinomial alto ou features colineares).

5. Basis Functions — O Truque Não-Linear

E se os dados não forem lineares? A **Basis Functions trick** resolve isso de forma genial: em vez de aplicar os coeficientes diretamente em x , primeiro **transformamos** x com funções $B_m(x)$. O modelo continua **linear nos coeficientes** — só as features mudam!

Analogia: pense em origami. Você dobra uma folha plana (transforma os dados), e dobras simples criam formas complexas. As dobras são as Basis Functions. O processo de encontrar os pesos é linear; a transformação é que traz a não-linearidade.

O modelo com M Basis Functions:

$$y_j = \sum_m a_m B_m(x_j) + \varepsilon_j \Rightarrow \mathbf{y} = \mathbf{B} \mathbf{a} + \varepsilon$$

$\mathbf{B} \in \mathbb{R}^{(K \times M)}$ (Basis Matrix) | $B_{jm} = B_m(x_j)$ | $\hat{\mathbf{a}} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{y}$ (mesma fórmula das Normal Equations).

Por que funciona? Os coeficientes a_m multiplicam as funções B_m , e a_m entram de forma **linear**. A não-linearidade está em $B_m(x)$, mas o sistema de equações para encontrar $\mathbf{a}$ permanece linear — logo, a solução analítica exata via Normal Equations ainda funciona.

Comparação de Basis Functions — mesmo dado, diferentes representações

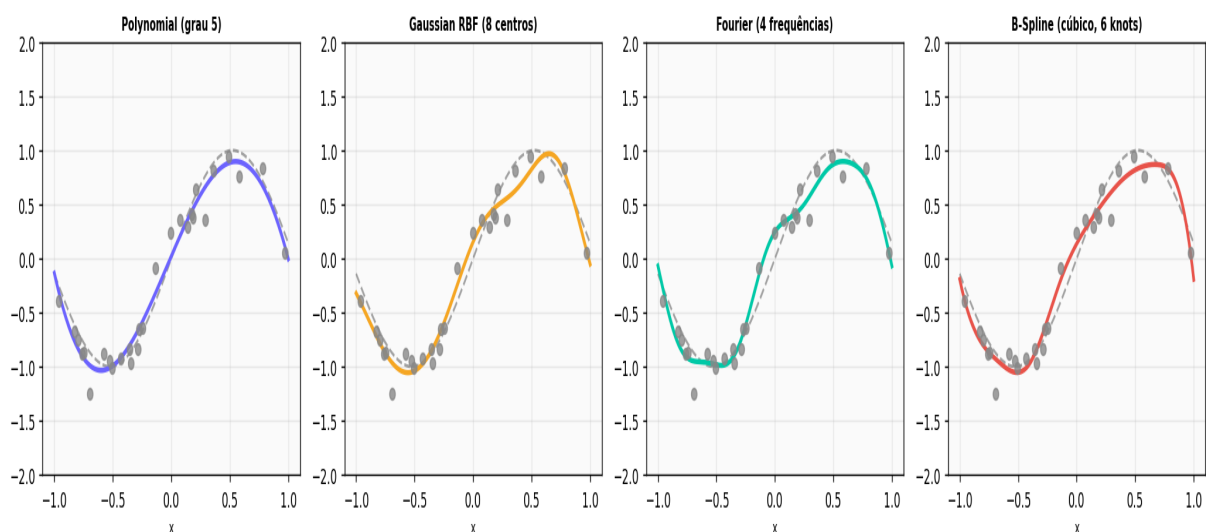


Figura 3 — Quatro tipos de Basis Functions aplicadas ao mesmo dataset: Polynomial, RBF, Fourier e B-Spline. Mesma fórmula $\hat{\mathbf{a}} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{y}$ para todas.

Por que isso é importante para Data Science?

General Linear Models (GLMs) são 'lineares' em um sentido muito específico: linear nos parâmetros $\mathbf{a}$, não nas features x . Isso é o que permite usar toda a teoria de regressão linear (Normal Equations, intervalos de confiança, testes de hipótese) mesmo com modelos não-lineares nas features.

Aplicação Prática em Machine Learning

No pipeline sklearn, `PolynomialFeatures().fit_transform(X)` constrói exatamente a Basis Matrix $\mathbf{B}$. Depois, `LinearRegression().fit(B, y)` resolve as Normal Equations. Todo pipeline não-linear do sklearn funciona nesse paradigma: transforme primeiro, regresse depois.

6. Polynomial Basis

A **Polynomial Basis** é a mais intuitiva: usamos potências de x como funções de base. Grau 1 é a reta clássica; grau 2 adiciona curvatura; graus altos capturam oscilações cada vez mais finas.

$$B_m(x) = x^m \quad m = 0, 1, 2, \dots, M$$

Cada coluna de $B_poly = [1, x, x^2, \dots, x^M]$ — uma potência por coluna

Construção em Python:

```
def polynomial_basis_matrix(x, degree):
    return np.column_stack([x**m for m in range(degree + 1)])

# Uma list comprehension cria cada coluna  $x^m$  e column_stack as empilha
```

Grau baixo → underfitting | Grau alto → overfitting (Runge's phenomenon)

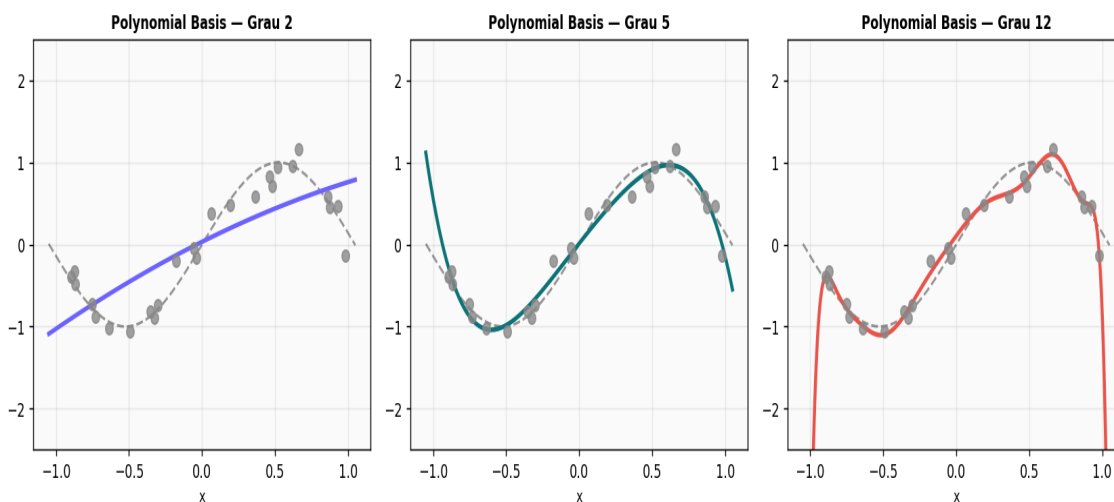


Figura 4 — Polynomial Basis: grau 2 (underfitting), grau 5 (bom equilíbrio), grau 12 (overfitting — Runge's phenomenon nas bordas).

Propriedade	Polynomial Basis
Vantagem	Simples de construir; solução analítica disponível
Desvantagem	Graus altos → overfitting e oscilações nas bordas
Localidade	NENHUMA — mudar um ponto afeta o polinômio inteiro
sklearn equivalente	PolynomialFeatures(degree=M)

Por que isso é importante para Data Science?

A Polynomial Basis é o primeiro passo conceitual, mas raramente a escolha final em produção. O fenômeno de Runge (oscilações nas bordas com grau alto) é um problema clássico que motiva o uso de Splines — que são polinômios locais e evitam o problema.

Aplicação Prática em Machine Learning

No sklearn: `from sklearn.preprocessing import PolynomialFeatures; poly = PolynomialFeatures(degree=5); X_poly = poly.fit_transform(X)`. Combine com Ridge para regularizar os coeficientes de alta potência, que tendem a ser muito grandes e instáveis.

7. Gaussian RBF Basis (Radial Basis Functions)

Imagine colocar **sinos gaussianos** espalhados pelo eixo x. O modelo é a soma ponderada desses sinos. Cada sino 'ativa' quando x está próximo de seu centro. Isso resolve o problema de **localidade** dos polinômios.

Analogia: pense em um time de especialistas. Cada especialista (RBF) domina uma região do espaço de entrada e contribui com sua expertise quando o ponto de consulta está na sua vizinhança. Os coeficientes a_m dizem o quanto cada especialista contribui.

$$B_m(x) = \exp\left(-\frac{(x - c_m)^2}{2\sigma^2}\right)$$

c_m : center (onde o sino está centrado) | σ : width (largura do sino)

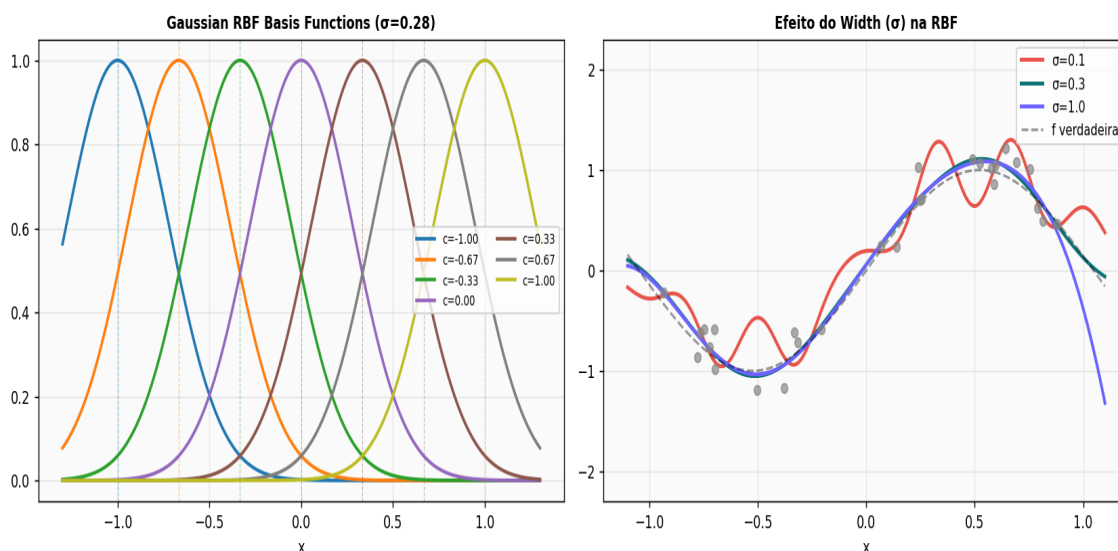


Figura 5 — Esquerda: 7 Gaussian RBFs centradas em posições equidistantes. Direita: efeito do width sigma — pequeno overfit, grande underfit.

Parâmetro	Valor pequeno	Valor grande
sigma (width)	Sinos estreitos → muito localizado → overfitting	Sinos largos → suave → underfitting
M (# centros)	Poucas bases → underfitting	Muitas bases → overfitting (sem regularização)

Por que isso é importante para Data Science?

RBFs são a base dos Support Vector Machines com kernel RBF (sklearn's SVC/SVR com kernel='rbf'). Também são fundamentais em redes neurais RBF e em Gaussian Processes. Entender o papel de center e width é entender o hiperparâmetro $\gamma = 1/(2\sigma^2)$ do SVM.

Aplicação Prática em Machine Learning

No sklearn: `sklearn.svm.SVR(kernel='rbf', gamma=0.5)`. O gamma controla exatamente sigma. Ao fazer Grid Search, você está otimizando a largura dos sinos RBF implícitos no kernel. sigma pequeno = modelo complexo = mais risco de overfitting.

8. Fourier Basis

Senos e cossenos são as basis functions naturais para **dados periódicos**. Som, ondas de rádio, temperaturas sazonais — todos têm periodicidade. A **Fourier Basis** decompõe qualquer sinal periódico em suas frequências constituintes.

Analogia: qualquer música pode ser decomposta em notas puras. Cada nota é uma frequência, e a música é a soma ponderada dessas notas. A Fourier Basis faz exatamente isso — os coeficientes a_m dizem quão forte é cada frequência no sinal.

$$B_{2k-1}(x) = \sin\left(\frac{2\pi kx}{T}\right), \quad B_{2k}(x) = \cos\left(\frac{2\pi kx}{T}\right)$$

$k = 1, 2, \dots, K$ frequências | $T = \text{período do sinal}$

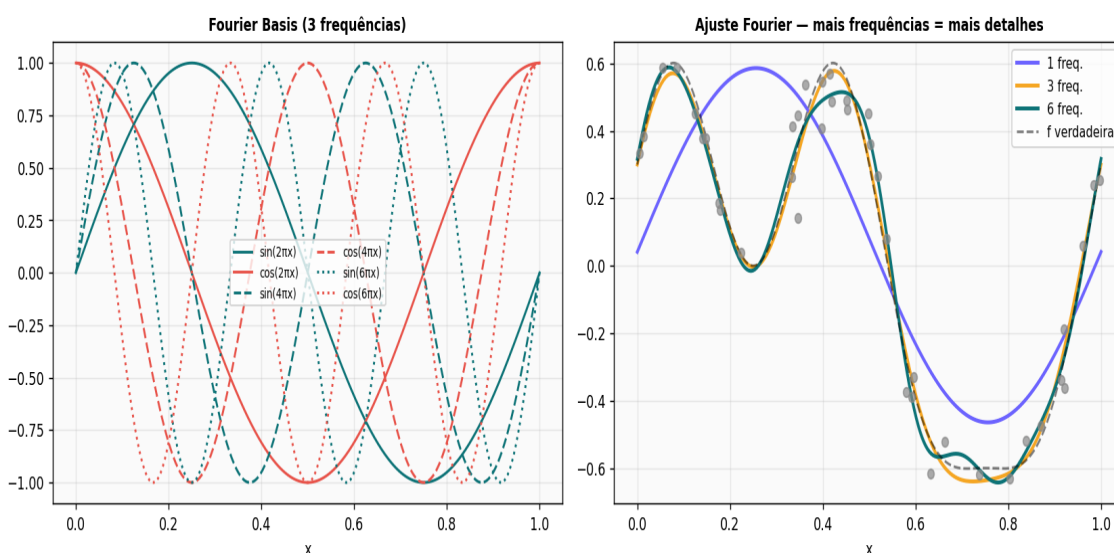


Figura 6 — Esquerda: basis functions de Fourier (senos azuis, cossenos vermelhos). Direita: mais frequências capturam padrões mais finos no sinal periódico.

Construção em Python:

```
def fourier_basis_matrix(x, n_freqs, periodo=1.0):
    cols = [np.ones_like(x)] # termo constante a0
    for k in range(1, n_freqs + 1):
        cols.append(np.sin(2*np.pi*k*x/periodo))
        cols.append(np.cos(2*np.pi*k*x/periodo))
    return np.column_stack(cols)
```

Por que isso é importante para Data Science?

Fourier Analysis é a espinha dorsal do processamento de sinais, compressão de imagens (JPEG usa DCT — uma variante de Fourier), análise de séries temporais sazonais, e até de redes neurais modernas (Fourier Neural Operators para PDEs). Para dados com periodicidade clara, é a representação mais eficiente que existe.

Aplicação Prática em Machine Learning

Para séries temporais sazonais (vendas mensais, temperatura diária), adicione features de Fourier ao seu pipeline antes de treinar qualquer modelo. No Prophet (biblioteca de forecasting do Facebook), as sazonalidades são modeladas exatamente com Fourier Basis. Você pode reproduzir esse comportamento manualmente com `fourier_basis_matrix`.

9. B-Splines — Compact Support e Cox-de Boor

B-Splines (Basis Splines) resolvem os problemas dos polinômios globais de forma elegante. A ideia: dividir o eixo x em segmentos com pontos de divisão chamados **Knots**, e ajustar polinômios separados em cada segmento, garantindo suavidade nas junções.

Analogia perfeita: é como montar um caminho em curva com peças de trilho de trem. Cada peça é levemente curvada, mas as junções são suaves — você não sente o impacto ao passar de uma para outra. Os Knots são os pontos de junção.

As duas propriedades fundamentais:

- **Compact Support:** cada $B_{\{m,p\}}(x)$ é exatamente zero fora de um intervalo pequeno. Mudar a_m afeta apenas a região local — ao contrário de polinômios globais que afetam o gráfico inteiro.
- **Partition of Unity:** em qualquer ponto x , as B-Splines ativas somam exatamente 1: $\sum_m B_{\{m,p\}}(x) = 1$. Isso garante estabilidade numérica e que os coeficientes têm interpretação de pesos.

Fórmula de Cox-de Boor (recursiva):

$$B_{m,0}(x) = 1 \text{ se } t_m \leq x < t_{m+1}, \quad 0 \text{ caso contrário}$$

Caso base: funções degrau constantes (grau 0)

$$B_{m,p}(x) = \frac{x - t_m}{t_{m+p} - t_m} B_{m,p-1}(x) + \frac{t_{m+p+1} - x}{t_{m+p+1} - t_{m+1}} B_{m+1,p-1}(x)$$

Recursão de grau p : combinação convexa de duas bases de grau $p-1$

B-Splines: basis functions, Partition of Unity, e Compact Support

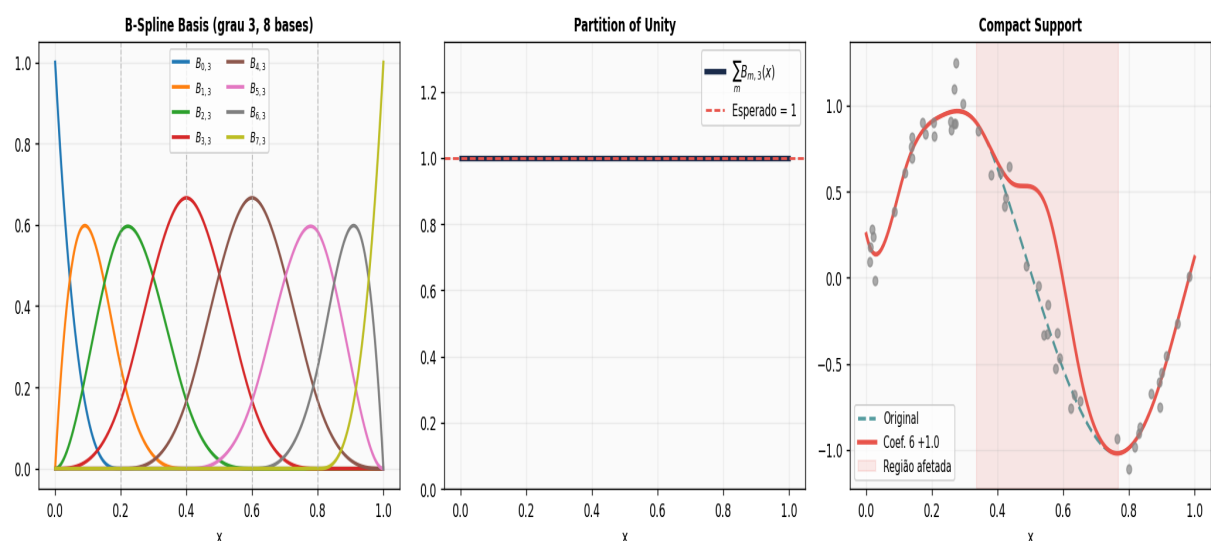


Figura 7 — Esquerda: B-Spline basis functions (grau 3). Centro: Partition of Unity (soma = 1 em todo ponto). Direita: Compact Support — perturbar um coeficiente afeta apenas a região local (área vermelha).

Propriedade	Polynomial	RBF	B-Spline
Compact Support	Não	Aproximado	Sim (exato)
Partition of Unity	Não	Não	Sim
Suavidade controlável	Não	Indiretamente	Sim (pelo grau)
Estabilidade numérica	Baixa	Média	Alta
sklearn/scipy	PolynomialFeatures	RBFSampler	make_lsqr_spline

Uso com scipy:

```
from scipy.interpolate import make_lsqr_spline, BSpline

knots_internos = np.linspace(0.1, 0.9, 8)
spl = make_lsqr_spline(x_treino, y_treino, knots_internos, k=3)
y_pred = spl(x_novo) # avaliação vetorizada
```

Por que isso é importante para Data Science?

B-Splines são o padrão-ouro em modelagem de curvas suaves. Toda vez que você vê uma curva bonita em gráficos científicos (spline interpolation, smoothing splines), provavelmente são B-Splines por baixo. Também são a base dos Generalized Additive Models (GAMs), onde cada feature tem uma spline independente.

Aplicação Prática em Machine Learning

Em Python, `scipy.interpolate.make_lsqr_spline` é a implementação estável e eficiente. Para GAMs completos (com regularização automática de suavidade), use a biblioteca `pygam`: `from pygam import LinearGAM`. GAMs são cada vez mais populares por combinar flexibilidade de B-Splines com interpretabilidade (você pode plotar o efeito de cada feature separadamente).

10. Resumo & Pipeline Unificado

Independentemente do tipo de Basis Function escolhida, o pipeline de ajuste é sempre o mesmo. A escolha da basis é uma decisão de design; a matemática de solução é universal.

$$\hat{a} = (B^T B)^{-1} B^T y$$

Passo 1: escolha as Basis Functions → Passo 2: construa B → Passo 3: resolva as Normal Equations

Conceito	Ideia Central	Equação Chave
Statistical Learning	Estrutura + ruído	$Y = f(x) + \text{epsilon}$
Linear Regression	Linear nos coeficientes	$y = Xa + \text{epsilon}$
Normal Equations	Solução analítica exata	$a_hat = (X^T X)^{-1} X^T y$
Basis Functions	Truque não-linear	$y_j = \sum_m a_m B_m(x_j)$
Polynomial Basis	Potências de x	$B_m(x) = x^m$
Gaussian RBF	Sinos locais	$B_m(x) = \exp(-(x-c_m)^2 / 2*\sigma^2)$
Fourier Basis	Frequências periódicas	$\sin(2*\pi*k*x/T), \cos(2*\pi*k*x/T)$
B-Splines	Suporte compacto	Cox-de Boor recursivo

Na próxima aula, veremos como escolher a complexidade certa do modelo (número de basis functions, grau do polinômio, sigma da RBF) usando **Bias-Variance Tradeoff**, **Cross-Validation** e **Regularização**. Os conceitos desta aula são o ponto de partida para tudo que vem a seguir.